

# BIMO-BE 사용자 매뉴얼 (Backend API)

문서 버전: 1.0

최종 업데이트: 2025-12-15

## 1. 프로젝트 제목, 팀 번호, 팀 구성원 소개

### 1.1 프로젝트 제목

- 프로젝트명**: BIMO (Backend: BIMO-BE)
- 한 줄 소개**: 비행 전/후에 필요한 정보를 한 곳에서 제공하고, 개인화된 비행 경험(리뷰, 타임라인, 알림)을 지원하는 **항공 동행 서비스 백엔드 API**

### 1.2 팀 번호

- 팀 번호**: (여기에 기입)

### 1.3 팀 구성원 소개

아래 표는 제출용 양식입니다. 팀에 맞게 수정하세요.

| 이름    | 학번    | 역할          | 담당 기능                       |
|-------|-------|-------------|-----------------------------|
| (이름1) | (학번1) | (예: 팀장/백엔드) | (예: 인증/JWT, Firebase 연동)    |
| (이름2) | (학번2) | (예: 백엔드)    | (예: 리뷰/항공사 통계)              |
| (이름3) | (학번3) | (예: 백엔드)    | (예: 항공편 검색(Duffel), 마이플라이트) |
| (이름4) | (학번4) | (예: 백엔드)    | (예: 오프라인/동기화, 알림(FCM))      |

## 2. 기존 서비스 문제 제기

### 2.1 문제 1: 비행 관련 정보가 여러 서비스에 흩어져 있음

사용자는 항공편 검색(예약 사이트), 공항/편명 확인(항공사 앱), 리뷰 확인(커뮤니티/블로그), 탑승 중 계획(메모/알람) 등을 **여러 앱**에서 처리합니다.

이 과정에서 다음 문제가 발생합니다.

- 같은 정보를 여러 번 입력해야 함 (공항 코드, 시간, 편명 등)
- 리뷰/평점/팁이 서비스마다 형식이 달라 비교가 어려움

- 비행 중에는 네트워크 불안정으로 정보 접근이 끊기기 쉬움

## 2.2 문제 2: 개인화된 "비행 중 타임라인" 부재

장거리 비행에서 중요한 것은 "언제 자고, 언제 먹고, 언제 집중할지" 같은 \*\*시간 계획\*\*입니다.

기존 서비스는 단순한 정보 제공은 가능해도, 사용자의 목표(수면/업무/휴식)에 맞춘 \*\*구체적인 타임라인\*\*을 자동으로 생성해주지 못합니다.

## 2.3 문제 3: 알림/리마인드가 사용자 맥락과 분리됨

알림은 유용하지만, 실제로는 "내 일정(마이플라이트)"과 연결되어야 의미가 있습니다.

많은 서비스는 알림을 제공하더라도, 사용자 데이터와 일관되게 관리되지 않거나(기기 변경 시 누락), 여러 디바이스에 대한 토큰 관리가 어렵습니다.

## 2.4 문제 4: 오프라인/네트워크 불안정 상황 대응 부족

공항/기내 환경에서는 네트워크가 불안정합니다. 이때 조회/저장 경험이 끊기면 서비스 만족도가 크게 떨어집니다.

BIMO-BE는 네트워크 모니터링과 로컬 캐시/큐를 통해 \*\*오프라인에서도 동작 가능한 구조\*\*를 제공합니다.

---

# 3. 구현된 프로젝트의 풀버전 GUI와 상세 매뉴얼(설치 방법과 사용법 기술)

## 3.1 "풀버전 GUI" 안내 (Swagger UI)

BIMO-BE는 백엔드 프로젝트이며, 실행 후 자동으로 제공되는 \*\*Swagger UI\*\*가 "풀버전 GUI" 역할을 합니다.

- \*\*Swagger UI\*\*: `http://localhost:8000/docs`
- \*\*ReDoc\*\*: `http://localhost:8000/redoc`
- \*\*OpenAPI JSON\*\*: `http://localhost:8000/openapi.json`

Swagger UI에서 버튼 클릭만으로 모든 API를 테스트할 수 있습니다(요청/응답 확인 포함).

---

## 3.2 설치(환경 구축) 방법 — Windows 기준

### 3.2.1 사전 준비물

- \*\*Python\*\*: 3.10 이상 권장

- **\*\*Git\*\***: 소스 코드 내려받기용
- **\*\*  
(필수)** Firebase 서비스 계정 키(JSON)\*\*: Firestore/Auth/FCM을 사용하기 위한 키
- **\*\*  
(선택)** Duffel API Key\*\*: 항공편/공항 검색 기능 사용 시 필요
- **\*\*  
(LLM 기능 사용 시 권장)** Ollama\*\*: 로컬 LLM 서버(비행 타임라인/LLM 챗 기능에 사용)

### 3.2.2 소스 코드 다운로드

- 1) 터미널(명령 프롬프트/PowerShell) 실행
- 2) 원하는 폴더로 이동 후 아래 실행

```
git clone https://github.com/BIMO-2025/BIMO-BE.git
cd BIMO-BE
```

### 3.2.3 가상환경 생성 및 의존성 설치

아래는 **\*\*가상환경(venv)\*\*** 사용을 권장합니다.

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

### 3.2.4 `.env` 파일 생성(필수)

프로젝트 루트('BIMO-BE/')에 `.env` 파일을 생성하고 아래를 참고하여 채웁니다.

```
# -----
# (필수) Firebase 설정
# -----
FIREBASE_SERVICE_ACCOUNT_KEY=./firebase_service_key.json

# -----
# (필수) JWT 설정 (우리 서비스 토큰)
# -----
API_SECRET_KEY=your-secret-key-here
API_TOKEN_ALGORITHM=HS256
API_TOKEN_EXPIRE_MINUTES=30
REFRESH_TOKEN_EXPIRE_DAYS=7

# -----
# (선택) Duffel API 설정 (항공편/공항 검색 사용 시)
# -----
DUFFEL_API_KEY=your-duffel-api-key-here
DUFFEL_ENVIRONMENT=test

# -----
# (권장) LLM(Ollama) 설정 (LLM 챗/비행 타임라인 사용 시)
# -----
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL_NAME=llama3.2:3b
```

**\*\*주의(매우 중요)\*\***

- `FIREBASE\_SERVICE\_ACCOUNT\_KEY` 경로가 틀리면 서버가 시작되지 않습니다.

- `API\_SECRET\_KEY`는 JWT 서명에 사용되므로 임의 문자열이지만, 외부에 노출되면 안 됩니다.

### 3.2.5 Firebase 서비스 계정 키 파일 준비(필수)

- 1) Firebase Console에서 서비스 계정 키(JSON)를 다운로드합니다.
- 2) 프로젝트 루트에 `firebase\_service\_key.json` 파일로 저장(혹은 다른 이름 사용 시 `.env` 경로 수정)

> 예: `BIMO-BE/firebase\_service\_key.json`

### 3.2.6 (선택) Ollama 설치 및 모델 준비(LLM 기능 사용 시)

LLM 챗(`/llm/chat`)과 비행 타임라인(`/wellness/flight-timeline`) 기능은 기본적으로 **Ollama**를 사용합니다.

- 1) Ollama 설치 후 실행
- 2) 모델 다운로드(예: llama3.2:3b)

```
ollama pull llama3.2:3b
```

Ollama 서버 기본 주소는 `http://localhost:11434` 입니다.

다른 주소/포트를 쓰면 `.env`의 `OLLAMA\_BASE\_URL`을 바꾸세요.

---

## 3.3 실행 방법

### 3.3.1 개발 모드 실행(자동 재시작)

아래 둘 중 하나로 실행합니다.

- **직접 실행**

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

- **Windows 스크립트 실행(권장)**

프로젝트 루트의 `run\_server.ps1` 또는 `run\_server.bat`를 실행합니다.

### 3.3.2 실행 확인

서버가 정상 실행되면 브라우저에서 아래로 접속합니다.

- `http://localhost:8000/` (루트)
  - `http://localhost:8000/docs` (Swagger UI)
-

### 3.4 사용 방법 (Swagger UI 기준, 따라하기 실습)

이 절은 "처음 보는 사람도 그대로 따라하면 동작 확인"이 목표입니다.

**\*\*순서대로 진행\*\***하세요.

---

#### 3.4.1 0단계: Swagger UI 열기

1) 브라우저에서 `http://localhost:8000/docs` 접속

2) 왼쪽에 태그별로 API 목록이 보입니다. 예)

- Authentication
  - User
  - Flights / Search
  - My Flights
  - Reviews
  - Airlines
  - LLM
  - Wellness
  - Notifications
  - Offline
- 

#### 3.4.2 1단계: 로그인(JWT 발급) — Authentication

소셜 로그인은 "클라이언트에서 받은 토큰"을 서버에 전달하는 구조입니다.

##### (A) Google / Apple 로그인

- **\*\*엔드포인트\*\***: `POST /auth/google/login`, `POST /auth/apple/login`
- **\*\*요청\*\***: Firebase ID Token 필요

Swagger UI에서:

1) `POST /auth/google/login` 펼치기

2) **\*\*Try it out\*\*** 클릭

3) Body에 아래 형태로 입력 후 **\*\*Execute\*\***

```
{  
  "token": "firebase_id_token_here",
```

```
    "fcm_token": "optional_fcm_device_token"
  }
```

성공하면 응답으로 아래가 반환됩니다.

- `access\_token`: 이후 API 호출에 사용하는 JWT
- `refresh\_token`: access token 갱신용
- `user`: uid/email/display\_name 등

## (B) Kakao 로그인

- **엔드포인트**: `POST /auth/kakao/login`
- **요청**: Kakao Access Token 필요

```
{
  "token": "kakao_access_token_here",
  "fcm_token": "optional_fcm_device_token"
}
```

---

### 3.4.3 2단계: 내 프로필 조회 — User

로그인 성공 후 발급받은 `access\_token`을 사용합니다.

- **엔드포인트**: `GET /user/profile`
- **헤더**: `Authorization: Bearer {access\_token}`

Swagger UI에서:

1) `GET /user/profile` 펼치기 → **Try it out**

2) `Authorization` 헤더 입력란이 없으면, Swagger UI의 "Authorize" 기능 대신 **Request headers**에 직접 넣어야 합니다.

- 현재 프로젝트는 일부 엔드포인트가 Header 파라미터로 `Authorization`을 읽습니다.

3) Execute 실행 → 사용자 프로필(UID 등) 확인

---

### 3.4.4 3단계: 항공편/공항 검색 — Search (Duffel)

이 기능은 **Duffel API Key**가 설정되어 있어야 동작합니다.

#### (A) 공항 IATA 코드 검색

- **엔드포인트**: `GET /search/airportIATACode?location=Seoul`

- **예시**: `location`에 "Seoul", "Tokyo Japan" 같은 문자열 입력

## (B) 특정 날짜/노선의 항공사(operating carrier) 검색

- **엔드포인트**: `POST /search/airlines`

예시 요청:

```
{
  "departure": "ICN",
  "arrive": "JFK",
  "departure_date": "2025-12-25"
}
```

---

### 3.4.5 4단계: 마이플라이트 저장/조회 — My Flights (Firestore)

마이플라이트는 사용자별로 Firestore 경로 `users/{userId}/myFlights/{myFlightId}`에 저장됩니다.

#### (A) 마이플라이트 생성

- **엔드포인트**: `POST /users/{user\_id}/my-flights`
- **인증**: Bearer Token 필요(토큰의 `sub`와 `{user\_id}`가 일치해야 함)

예시 Body(직항):

```
{
  "segments": [
    {
      "operating_carrier": "KE",
      "flight_number": "KE901",
      "duration": "14H30M",
      "departure": { "iata_code": "ICN", "at": "2025-12-25T13:45:00Z" },
      "arrival": { "iata_code": "JFK", "at": "2025-12-25T18:20:00Z" }
    }
  ],
  "departureTime": "2025-12-25T13:45:00Z",
  "arrivalTime": "2025-12-25T18:20:00Z",
  "status": "scheduled",
  "departureAirport": "ICN",
  "arrivalAirport": "JFK",
  "hasStopover": false
}
```

성공 시 응답 예:

```
{ "id": "생성된_비행기록_ID", "message": "비행 기록이 생성되었습니다." }
```

#### (B) 마이플라이트 목록 조회

- **엔드포인트**: `GET /users/{user\_id}/my-flights?status=scheduled&limit=20`

### (C) 마이플라이트 단건 조회/수정/삭제

- `GET /users/{user\_id}/my-flights/{flight\_id}`
  - `PUT /users/{user\_id}/my-flights/{flight\_id}`
  - `DELETE /users/{user\_id}/my-flights/{flight\_id}`
- 

### 3.4.6 5단계: 비행 타임라인 생성 — Wellness (LLM/Ollama)

이 기능은 LLM을 사용하며, 기본적으로 **Ollama**가 필요합니다.

#### (A) 직접 타임라인 생성

- **엔드포인트**: `POST /wellness/flight-timeline`

예시 Body:

```
{
  "origin": "ICN",
  "destination": "JFK",
  "departure_time": "2025-12-25T13:45:00",
  "arrival_time": "2025-12-25T18:20:00",
  "seat_class": "ECONOMY",
  "flight_goal": "SLEEP_FOCUS",
  "total_duration": "14h 35m"
}
```

응답에는 다음이 포함됩니다.

- `recommendation\_message`: 사용자 메시지
- `timeline\_events`: 이륙/식사/수면/자유시간/착륙 등 이벤트(시작/종료/표시 시간 포함)

#### (B) 마이플라이트 기반 타임라인 생성

- **엔드포인트**: `POST /wellness/users/{user\_id}/my-flights/{flight\_id}/timeline`
  - **인증**: Firebase 토큰 검증(요청 헤더 토큰 필요)
- 

### 3.4.7 6단계: 리뷰 기능 — Reviews / Airlines

리뷰는 항공사 페이지/상세 페이지에서 사용됩니다.

#### (A) 항공사 리뷰 페이지 조회

- **엔드포인트**: `GET /airlines/{airline\_code}/reviews?sort=latest&limit=20&offset=0`



## (B) 항공사 상세 리뷰(필터/정렬) 조회

- **엔드포인트**: `GET /reviews/detailed/{airline\_code}`
- **필터 예시**: `seat\_class=이코노미`, `min\_rating=4`, `photo\_only=true`

## (C) 리뷰 작성/수정/삭제(인증 필요)

- `POST /reviews` (생성)
- `PUT /reviews/{review\_id}` (수정)
- `DELETE /reviews/{review\_id}` (삭제)

리뷰 생성/수정/삭제 시, 백그라운드에서:

- 항공사 통계 업데이트
- BIMO 요약 업데이트

가 수행됩니다.

## (D) 리뷰 "좋아요" 추가

- `POST /reviews/{review\_id}/like`

---

### 3.4.8 7단계: 이미지 업로드(압축→Base64) — Uploads

리뷰 작성 시 이미지가 필요하다면, 이 API로 **이미지를 800x800/품질85로 압축한 뒤 Base64 Data URL**로 받을 수 있습니다.

- **엔드포인트**: `POST /uploads/images/base64`
- **형식**: multipart/form-data

응답 예:

```
{
  "success": true,
  "images": ["data:image/jpeg;base64,..."],
  "count": 1,
  "message": "1개의 이미지가 성공적으로 처리되었습니다."
}
```

---

### 3.4.9 8단계: 알림(FCM) — Notifications

FCM 알림은 사용자 계정에 저장된 FCM 토큰들을 대상으로 전송됩니다.

#### (A) 로그인 시 FCM 토큰 저장

로그인 요청(`/auth/\*/login`) Body에 `fcm\_token`을 넣으면 서버가 사용자 문서에 토큰을 저장합니다(중복 방지).

#### (B) 토큰 업데이트/제거

- `POST /notifications/token/update`
- `POST /notifications/token/remove`

#### (C) 알림 전송

- `POST /notifications/send`
- **\*\*인증\*\***: Bearer Token 필요

예시 Body:

```
{
  "title": "출발 시간 알림",
  "body": "곧 출발 시간입니다. 체크인을 완료해주세요.",
  "data": { "type": "flight_reminder" }
}
```

---

### 3.4.10 9단계: 오프라인/동기화 상태 — Offline

네트워크 상태 및 동기화 큐 상태를 확인할 수 있습니다.

- `GET /offline/status` : 네트워크 상태/대기 큐 개수 등
  - `POST /offline/sync` : 수동 동기화 실행
- 

## 3.5 자주 발생하는 문제 해결(트러블슈팅)

### 3.5.1 서버가 시작하자마자 종료됨(Firebase 관련)

원인:

- `.env`에 `FIREBASE\_SERVICE\_ACCOUNT\_KEY`가 없거나 경로가 틀림
- 서비스 계정 JSON 파일이 유효하지 않음

해결:

- `.env` 경로 확인(예: `./firebase\_service\_key.json`)
- 파일이 실제로 존재하는지 확인

### 3.5.2 항공편/공항 검색이 실패함(Duffel 관련)

원인:

- `.env`에 `DUFFEL\_API\_KEY`가 없음

해결:

- Duffel 키를 발급받아 `.env`에 추가 후 서버 재시작

### 3.5.3 LLM/타임라인이 실패함(Ollama 관련)

원인:

- Ollama가 실행 중이 아님
- 모델이 설치되지 않음
- `.env`의 `OLLAMA\_BASE\_URL`이 실제 주소와 다름

해결:

- Ollama 실행 확인
- `ollama pull llama3.2:3b` 실행
- `.env` 확인 후 서버 재시작

---

## 4. 기대 효과

### 4.1 사용자 관점

- **\*\*비행 정보·리뷰·계획·알림의 통합\*\***: 여러 앱에서 하던 작업을 한 흐름으로 연결
- **\*\*개인화된 비행 중 타임라인 제공\*\***: 목표(수면/업무/휴식)에 맞춘 일정 자동 생성
- **\*\*네트워크 불안정 상황 대응\*\***: 오프라인 캐시/큐를 통한 안정적인 사용자 경험

### 4.2 서비스/개발 관점

- **\*\*명확한 API 문서(Swagger UI)\*\***: 프론트엔드/백엔드 협업 비용 감소
- **\*\*모듈화된 기능 구조\*\***: auth/users/flights/reviews/wellness/notifications/offline 등 기능 단위 확장 용이

- **외부 서비스 연동 기반**: Duffel(항공편 검색), Firebase(인증/DB/FCM), Ollama(LLM)로 확장성 확보
- 

## 5. Github URL

- **Repository**: ``https://github.com/BIMO-2025/BIMO-BE.git``